EduSmart School ERP ��� Backend Roadmap

EduSmart School ERP — Backend Roadmap

> Frontend architecture (Flutter, feature-first + Riverpod) already design ho chuki hai. Ye document uska backend counterpart hai — abhi tak sab modules dummy data pe chal rahe hain, isko real backend se connect karne ka step-by-step plan.

---

1. Scope — Codebase se nikle Real Entities

Har module ke models scan kiye — ye backend mein banne wale core domains hain (dummy data classes already isi shape mein hain, seedha reference ban sakte hain):

Module	Core Entities (already Flutter models mein maujood)
Students	Student, StudentStatus, RiskLevel
Teachers	Teacher, Department
Attendance	AttendanceRecord, AttendanceStatus, AttendanceTrendPoint
Fees	FeeHead, Installment, FeeTransaction, ExpenseEntry, IncomeEntry, AuditLogEntry
Examination	Examination, ExamSubjectSchedule, QuestionItem, QuestionPaperSet, EvaluationRecord, ResultRecord,

Module	Core Entities (already Flutter models mein maujood)
	RevaluationRequest, MalpracticeIncident, SeatAssignment, Invigilator
HR & Payroll	Employee, Department, Designation, LeaveRequest, Shift, PayrollRecord, SalaryStructure, LoanAdvance, TaxRecord, JobOpening, Candidate
Transport	TripRow, RouteSummaryRow, AlertData
Hostel	HostelBlock, Room, Bed, StaffMember, Visitor, Complaint, MaintenanceRequest, LeaveRequest, Payment
Inventory	InventoryItem, StockActivity, CategoryData
Parents (portal)	ChildRecord, FeeSummary, AcademicSummary, HomeworkItem, TimetableSlot, TransportInfo, LibraryBook, ChatMessage
Settings	App-level config

Observation: `Student` class already 3 modules mein alag-alag shape mein duplicate hai (fees, examination, hostel, students, parents) — backend mein ye **ek hi Student table/entity** honi chahiye jise sab modules refer karein (foreign key se), taaki data consistent rahe.

---

2. Tech Stack Recommendation

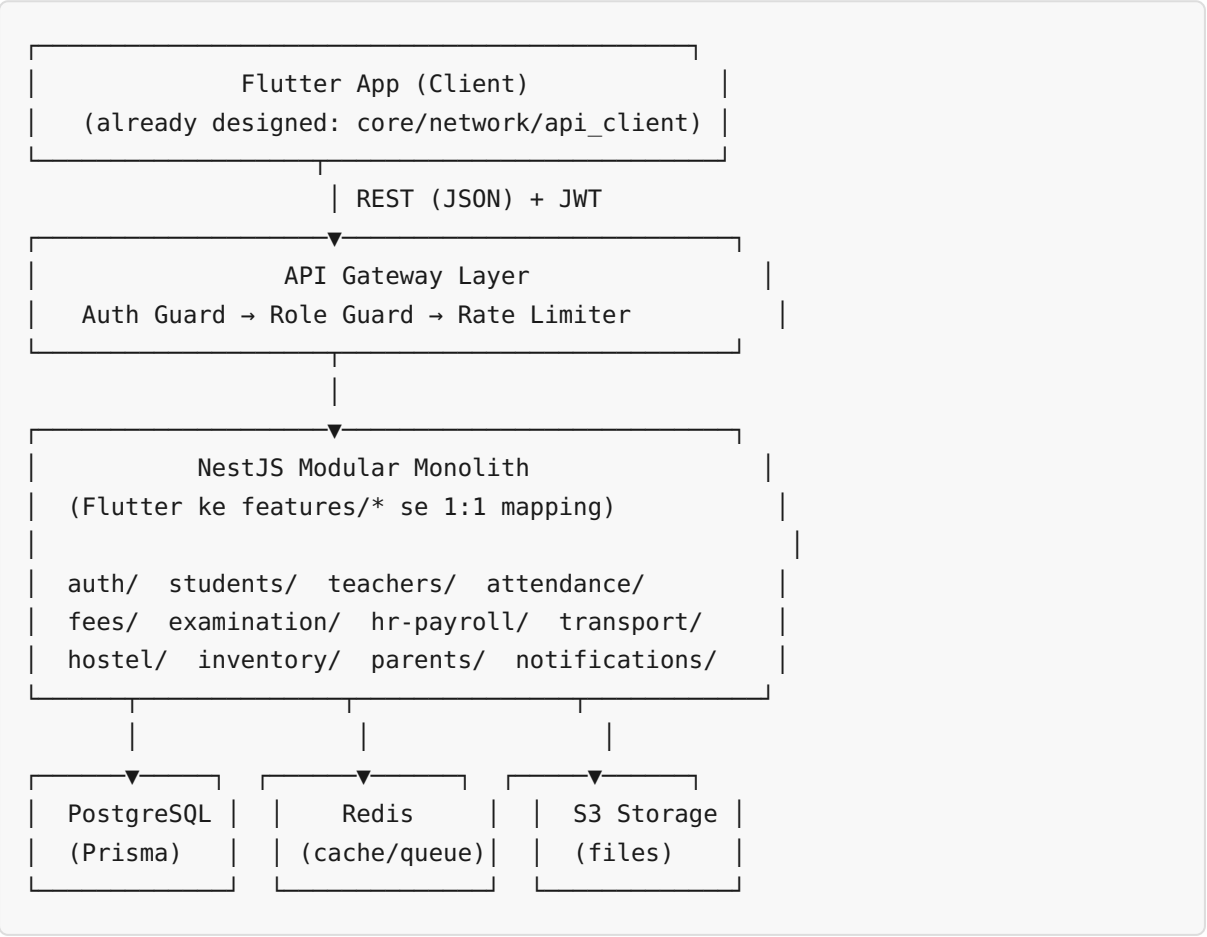
Layer	Recommendation	Reasoning
API Framework	**NestJS (Node.js + TypeScript)**	Modular architecture (modules/controllers/services) — Flutter ke feature-first structure se directly maps hota hai. Strong typing, DI built-in, bada ecosystem.

Layer	Recommendation	Reasoning
Database	**PostgreSQL**	Fees, payroll, exam results — sab strongly relational data hai (foreign keys, transactions zaroori). Relational DB best fit.
ORM	**Prisma**	Type-safe queries, migrations built-in, NestJS ke saath achha integrate hota hai.
Auth	**JWT (access + refresh token)** + role-based guards	Flutter models mein already `UserRole` / `AppRole` / `Permission` enums maujood hain — inhi ko backend RBAC ka base banayenge.
File Storage	**S3-compatible storage** (AWS S3 / Cloudflare R2 / MinIO for self-host)	PDF receipts, question papers, ID cards, documents — sab yahan store honge.
Caching/Queue	**Redis**	Session cache, rate-limiting, background jobs (payroll processing, bulk notifications) ke liye.
Background Jobs	**BullMQ** (Redis-based)	Payroll batch processing, bulk SMS/email/ notification, PDF generation jaisi heavy/ async tasks ke liye.
Real-time (optional, later)	**WebSockets (Socket.io)**	Parent-teacher chat, live notifications ke liye — Phase 2+ mein.
API Docs	**Swagger/OpenAPI** (NestJS built-in support)	Frontend team ke liye auto-generated API reference.
Hosting	Railway / Render / AWS (ECS+RDS) — koi bhi, containerized (Docker)	Scale karne layak, environment-based deploy.

> Agar team already Python/Django ya Firebase mein comfortable hai to wo bhi viable options hain — Firebase Firestore (NoSQL) fast prototyping ke liye achha hai but fees/payroll/exam jaise complex relational reports ke liye Postgres zyada reliable rahega long-term.

---

3. High-Level Backend Architecture



“Modular Monolith” kyun, microservices nahi? Abhi scale/team size ke hisaab se ek hi deployable app — module boundaries strict rakhne se (jaisa Flutter side pe bhi hai) — future mein zaroorat pade to individual modules ko microservice mein extract karna easy rahega.

---

4. Backend Module ↔ Flutter Feature Mapping

Har backend module, Flutter ke `features/<module>` se exactly map karta hai — isse frontend/backend dono team same mental model use karenge:

```
src/
├── auth/                   # login, refresh token, role/permission
├── students/               # single source of truth — sab modules yahan se
refer karte hain
├── teachers/
├── attendance/             # student + staff dono ke liye
├── fees/                   # fee-heads, installments, transactions, expense/
income, audit-log
├── examination/            # exams, question-bank, seating, evaluation,
results, revaluation
├── hr-payroll/             # employees, payroll, leave, recruitment
├── transport/              # routes, trips, alerts
├── hostel/                 # blocks, rooms, beds, complaints, visitors
├── inventory/              # items, stock activity, categories
├── parents/                # aggregation layer — students/fees/attendance/
examination ka combined view child ke liye
├── notifications/          # shared — sabhi modules yahan se notification
bhejenge
└── common/                 # shared DTOs, guards, interceptors, filters
```

---

5. Auth & Role Design (RBAC)

Flutter code mein already 3 alag role enums mile (`UserRole` in fees, `AppRole` / `Permission` in examination) — backend mein inhe **ek unified role system** mein consolidate karna hai:

Unified Roles: `super_admin` , `admin` , `principal` , `teacher` , `accountant` , `cashier` , `hr_staff` , `hostel_warden` , `transport_manager` , `inventory_manager` , `parent` , `student`

Design: - Ek `User` table (login credentials) → linked to ek `Role` + optionally linked to `Student` / `Employee` / `Parent` record (jisse wo user actually corresponds karta hai) - Permission granularity: module-level (`fees:read` , `fees:write` , `examination:approve_results` , etc.) — examination module ke `Permission` enum ka pattern isi tarah generalize karenge - JWT payload mein: `userId` , `role` , `permissions[]` - Har backend module ke controller endpoints par `@Roles()` / `@Permissions()` decorator guard

6. API Design Conventions

Aspect	Convention
Style	REST, JSON
Base path	`/api/v1/<module>/<resource>`
Auth	`Authorization: Bearer <JWT>`
Pagination	`?page=1&limit=20` → response mein `{ data, total, page, totalPages }`
Filtering	Query params (`?status=active&class=10`)
Error format	`{ statusCode, message, error, timestamp, path }` (consistent across sab modules)
Versioning	URL-based (`/v1/`) — future breaking changes ke liye `/v2/`
Soft delete	Zyada tar entities (Student, Employee, InventoryItem) mein hard-delete nahi, `deletedAt` field

7. Phase-wise Backend Roadmap

Phase B0 — Foundation (1-2 weeks)

- NestJS project setup (modular structure, ESLint/Prettier)
- PostgreSQL + Prisma setup, connect
- `common/` — global exception filter, response interceptor, validation pipe (`class-validator`)
- Docker + docker-compose (app + postgres + redis local dev ke liye)
- CI pipeline (lint + test on PR)

Phase B1 — Auth + Core Identity (1-2 weeks)

- `auth` module: register/login/refresh-token/logout
- Unified `User`, `Role`, `Permission` schema
- `students` module: Student CRUD (single source of truth — sab modules isi ko reference karenge)
- `teachers` / `employees` base schema

Phase B2 — High-Value Modules (3-4 weeks)

Priority wahi jo frontend roadmap mein bhi thi: - **fees**: fee-heads, installments, transactions, receipts (PDF generate), reports - **attendance**: mark/bulk-import/export, trends - **examination**: exam scheduling, results — pehle core flow (schedule → marks entry → result declare), advanced features (revaluation, malpractice) baad mein

Phase B3 — Remaining Modules (4-6 weeks)

- `hr-payroll`, `transport`, `hostel`, `inventory` — ek-ek karke, same pattern (module → schema → CRUD → business logic → tests)

Phase B4 — Parent Portal Aggregation (1-2 weeks)

- `parents` module — koi naya data nahi, balki `students` / `fees` / `attendance` / `examination` se combined read-only view banata hai ek child ke liye
- Multi-child support (ek parent ke multiple bacche)

Phase B5 — Cross-cutting Features (2-3 weeks)

- `notifications` module (in-app + push hook, email/SMS provider integrate)
- File upload/storage (S3) — receipts, ID cards, question papers, documents
- Background jobs (payroll batch processing, bulk notification, report generation)
- Audit logging — sabhi modules ke liye shared `AuditLogEntry` (fees module mein already concept hai)

Phase B6 — Hardening & Launch (2-3 weeks)

- Rate limiting, input sanitization, security headers
- Load testing critical endpoints (fee payment, attendance bulk mark, result declare)
- API documentation finalize (Swagger)
- Staging → Production deploy pipeline
- Monitoring/alerting (e.g. Sentry for errors, uptime monitoring)

---

8. Step-by-Step Execution Checklist

1. ☐ **Decision lo: single-school ya multi-school (SaaS)?** — Agar multi-school, sab tables mein `schoolId` Day 1 se add karo (Section 9 dekho — ye sabse critical decision hai, isse pehle koi schema mat likho).
2. ☐ Repo setup — NestJS monorepo (ya standalone) + Docker + Postgres + Redis local environment
3. ☐ `common/` — exception filter, response interceptor, validation pipe, logger
4. ☐ Prisma schema — `User`, `Role`, `Permission` (unified auth base)
5. ☐ `auth` module — register/login/JWT refresh/logout endpoints + guards
6. ☐ Prisma schema — `Student` (single source of truth entity)
7. ☐ `students` module — CRUD + role-guarded endpoints
8. ☐ Prisma schema — `Employee`, `Department`, `Designation` (teachers + hr-payroll dono isko share karenge)
9. ☐ `teachers` module — CRUD
10. ☐ Prisma schema — `FeeHead`, `Installment`, `FeeTransaction`, `ExpenseEntry`, `IncomeEntry`
11. ☐ `fees` module — collection, receipt PDF (S3 upload), reports endpoints

12. ☐ Prisma schema — `AttendanceRecord` (student + staff dono ke liye generic banao)
13. ☐ `attendance` module — mark/bulk-import/export/trends endpoints
14. ☐ Prisma schema — Examination domain (Exam, Schedule, QuestionBank, Evaluation, Result)
15. ☐ `examination` module — core flow (schedule → marks → result) pehle, advanced flows baad mein
16. ☐ **Anonymous evaluation system** — encrypted anonymous-ID↔student mapping table + QR/barcode answer-book lifecycle + `revealIdentity` permission-gated + audit-logged action (Section 9)
17. ☐ Remaining modules (`hr-payroll` , `transport` , `hostel` , `inventory`) — same repeat pattern
18. ☐ `parents` aggregation module — read-only combined views
19. ☐ `notifications` module — normal queued channel + **separate high-priority/real-time channel for hostel SOS alerts** (Section 9)
20. ☐ S3 file service + background job queue (Redis/BullMQ)
21. ☐ Security hardening pass (rate limit, sanitization, headers) + load test critical endpoints
22. ☐ Swagger docs finalize + staging deploy
23. ☐ Production deploy + monitoring setup

---

9. Domain-Specific Gaps (Audit Round 2)

Codebase ko aur gehraai se dekhne par 4 important cheezein mili jo generic CRUD se aage hain — inko roadmap mein explicitly plan karna zaroori hai:

Gap	Codebase mein kya mila	Backend implication
Anonymous evaluation / blind grading security	`examination/services/security_services.dart` mein already ek proper `RbacService` (fine-grained permission matrix, super-detailed — `examController`, `examCoordinator` roles alag-alag 20+ permissions ke saath) + `AnonymousBook` system hai jo `sha256` + `Random.secure()` se non-sequential anonymous IDs banata hai taaki evaluators ko student identity na pata chale.	Ye sirf UI logic nahi — backend mein **security-critical** hai. Anonymous-ID ↔ real-student mapping table **encrypted at rest** honi chahiye, aur "reveal identity" jaisa action sirf specific permission (`revealIdentity`) + audit-logged hona chahiye. Isko normal CRUD ki tarah treat nahi kar sakte — Phase B2 (examination) mein isko dedicated sub-task banana hoga.
QR/Barcode generation	`barcode_center_screen.dart` — `qr_flutter` se answer-book QR codes generate karta hai, tracking (assigned/used/unused count) ke saath.	Backend mein QR data encode/decode + answer-book lifecycle status (generated → assigned → submitted → evaluated) track karna hoga — ye `examination` module ka apna sub-domain hai, `Section 7 Phase B2` mein add karna chahiye tha explicitly.
Multi-tenancy / Single-school assumption	Poore codebase mein kahin `school_id` ya `tenant_id` jaisa field nahi mila — matlab abhi ye **single-school** app hai.	**Critical decision jo roadmap mein missing tha:** Kya ye ek hi school ke liye hai, ya aage multiple schools (SaaS product) ko serve karna hai? Agar multi-school plan hai, to **har table mein `schoolId` column Day 1 se add karna padega** — baad mein retrofit karna bahut costly hota hai (migration + data isolation dono). Ye

Gap	Codebase mein kya mila	Backend implication
		decision Phase B0 se pehle lena zaroori hai.
Real-time / urgent alerts	Hostel module ka `emergency_sos_card.dart` — Fire/Medical/Security alert buttons, jinhe **turant** warden/ security tak pahunchna chahiye.	Ye normal `notifications` module (Phase B5, queued/async) se alag hai — SOS jaisa alert **real-time push** (WebSocket ya high-priority FCM channel) chahiye, delay-tolerant nahi. Roadmap mein `notifications` module ke andar "priority/ urgent channel" explicitly add karna chahiye.
Cross-module search	`fees/features/search/command_palette.dart` — students + fee-transactions dono ko ek saath search karta hai (command-palette style).	Agar aage sab modules (students, employees, inventory) is tarah ka global search chahenge, to simple `LIKE` query kaafi na ho scale par — **Postgres full-text search** (`tsvector`) ya baad mein Elasticsearch/ Meilisearch consider karna, Phase B5/B6 mein note karna chahiye.

Sabse important open question: Multi-tenancy wala decision (single school vs multi-school SaaS) — iska jawab pura DB schema design badal deta hai, isliye Phase B0 shuru karne se pehle ye confirm karna chahiye.

---

10. Frontend-Backend Sync Point

Jab bhi koi backend module ready ho jaye, Flutter side `core/network/api_client.dart` (jo pehle se design ho chuka hai) use karke us module ki `RepositoryImpl` ko dummy data se real API call mein switch kar dena hai — baaki UI/Provider code same rahega (Section 7, pehle document — Data Layer design isi ke liye banaya tha).

Agla step: bataiye — main Phase B0 se actual NestJS + Prisma project scaffold kar doon (folder structure + starter code), ya pehle kisi ek module (jaise `fees` ya `students`) ka full Prisma schema + API endpoints design likh doon?